

Lab 2 Report:

Iris Classification with Regression

Name:

In [3]: `# Import necessary packages`

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt

import torch
```

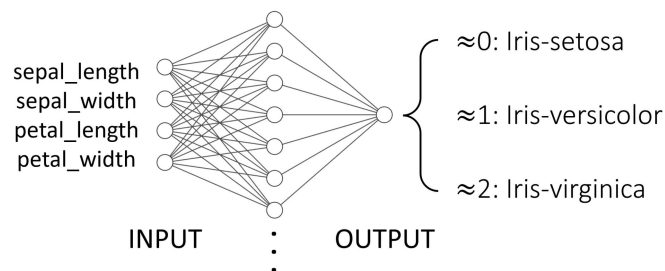
In [4]: `from IPython.display import Image # For displaying images in colab jupyter cell`

In [5]: `Image('lab2_exercise1.PNG', width = 1000)`

Out[5]:



Exercise 1: Iris Classification using Regression



In this exercise, you will train a neural network with a single hidden layer consisting of linear neurons to perform regression on iris datasets.

Your goal is to achieve a training accuracy of >90% under 50 epochs.

You are free to experiment with different data normalization methods, size of the hidden layer, learning rate and epochs.

You can round the output value to an integer (e.g. 0.34 -> 0, 1.78 -> 2) to compute the model accuracy.

Demonstrate the performance of your model via plotting the training loss and printing out the training accuracy. 42

Prepare Data

In [7]: `from sklearn.datasets import load_iris`

```
# iris dataset is available from scikit-learn package
iris = load_iris()

# Load the X (features) and y (targets) for training
X_train = iris['data']
y_train = iris['target']

# Load the name labels for features and targets
feature_names = iris['feature_names']
```

```
names = iris['target_names']
```

```
# Feel free to perform additional data processing here (e.g. standard scaling)
```

In [8]: *# Print the first 10 training samples for both features and targets*

```
print(X_train[:10, :], y_train[:10])
```

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]] [0 0 0 0 0 0 0 0 0 0]
```

In [9]: *# Print the dimensions of features and targets*

```
print(X_train.shape, y_train.shape)
```

```
(150, 4) (150,)
```

In [10]: *# feature_names contains name for each column in X_train*
For targets, 0 -> setosa, 1 -> versicolor, 2 -> virginica

```
print(feature_names, names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'] ['setosa' 'versicolor' 'virginica']
```

In [11]: *# We can visualize the dataset before training*

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
# enumerate picks up both the index (0, 1, 2) and the element ('setosa', 'versicolor', 'virginica')
# Loop 1: target = 0, target_name = 'setosa'
# Loop 2: target = 1, target_name = 'versicolor' etc
```

```
for target, target_name in enumerate(names):
```

```
    # Subset the rows of X_train that fall into each flower category using boolean indexing
    X_plot = X_train[y_train == target]
```

```
    # Plot the sepal length versus sepal width for the flower category
    ax1.plot(X_plot[:, 0], X_plot[:, 1], linestyle='none', marker='o', label=target_name)
```

```
# Label the plot
```

```
ax1.set_xlabel(feature_names[0])
ax1.set_ylabel(feature_names[1])
ax1.axis('equal')
ax1.legend()
```

```
# Repeat the above process but with petal length versus petal width
```

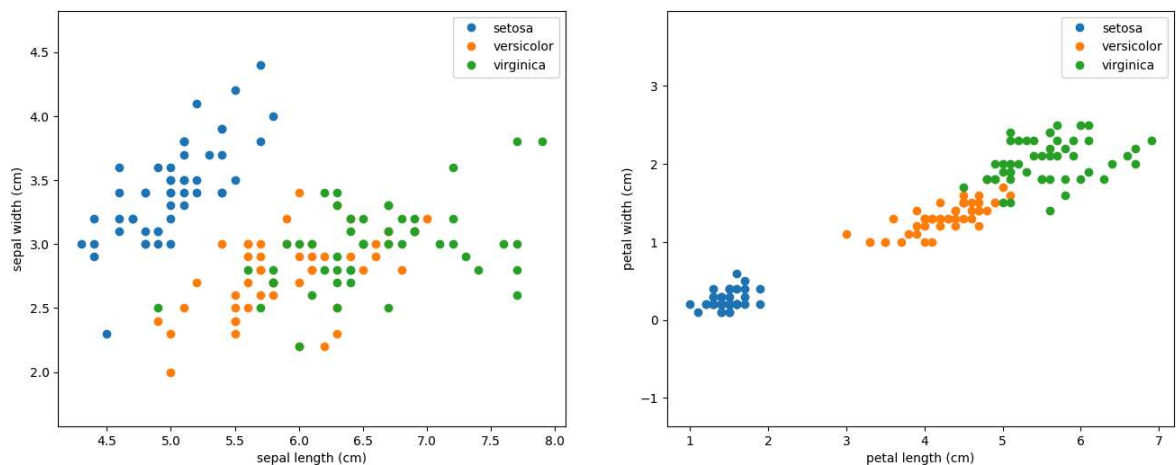
```
for target, target_name in enumerate(names):
```

```
    X_plot = X_train[y_train == target]
```

```
    ax2.plot(X_plot[:, 2], X_plot[:, 3], linestyle='none', marker='o', label=target_name)
```

```
ax2.set_xlabel(feature_names[2])
ax2.set_ylabel(feature_names[3])
ax2.axis('equal')
ax2.legend()
```

Out[11]: <matplotlib.legend.Legend at 0x15d0141c3d0>



Define Model

```
In [13]: class irisClassification(torch.nn.Module):

    def __init__(self, input_dim, output_dim, hidden_dim):

        super(irisClassification, self).__init__()

        self.layer1 = torch.nn.Linear(input_dim, hidden_dim) # YOUR CODE HERE # A
        self.layer2 = torch.nn.Linear(hidden_dim, output_dim) # YOUR CODE HERE #

    def forward(self, x):

        out = self.layer1(x) # YOUR CODE HERE
        out = self.layer2(out)

        return out
```

Define Hyperparameters

```
In [15]: model = irisClassification(input_dim = 4, output_dim = 1, hidden_dim = 2) # YOUR

learning_rate = 0.01 # YOUR CODE HERE
epochs = 35 # YOUR CODE HERE, This has to be <50

# We will use gradient descent for our optimizer and Mean Squared Error Loss fun
loss_func = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = learning_rate)
```

Identify Tracked Values

```
In [17]: # follow models performance over each epoch. Identify a metric and track it over
train_loss_list = []
```

```
# YOUR CODE HERE
```

Train Model

```
In [19]: X_train = torch.from_numpy(X_train).float()
y_train = torch.from_numpy(y_train).float()
y_train = y_train.unsqueeze(1)

epoch_list = []
for epoch in range(epochs):

    optimizer.zero_grad() # Empty the gradient buffer so each learning event per

    outputs = model(X_train) # Forward pass the inputs through the network to pr
    print(outputs.shape)
    loss = loss_func(outputs, y_train) # Compute the loss via comparing the outp

    train_loss_list.append(loss.item()) # Save the loss value to train_loss_list

    loss.backward() # Compute how much changes to be made to weights/biases

    optimizer.step() # Update the weights/biases according to Learning rate

    print('epoch {}, loss {}'.format(epoch, loss.item()))
    # print(epoch, loss.item())
    epoch_list.append(epoch)
    # print(X_train.shape)
    # print(y_train.shape)
    # print()
    # train_loss_list.append(loss.item())
    # YOUR CODE HERE
```

```
torch.Size([150, 1])
epoch 0, loss 1.6376800537109375
torch.Size([150, 1])
epoch 1, loss 0.5516006350517273
torch.Size([150, 1])
epoch 2, loss 0.4694954454898834
torch.Size([150, 1])
epoch 3, loss 0.43849924206733704
torch.Size([150, 1])
epoch 4, loss 0.41108885407447815
torch.Size([150, 1])
epoch 5, loss 0.3856584131717682
torch.Size([150, 1])
epoch 6, loss 0.36200758814811707
torch.Size([150, 1])
epoch 7, loss 0.3399885594844818
torch.Size([150, 1])
epoch 8, loss 0.31947189569473267
torch.Size([150, 1])
epoch 9, loss 0.3003430664539337
torch.Size([150, 1])
epoch 10, loss 0.28250008821487427
torch.Size([150, 1])
epoch 11, loss 0.26585152745246887
torch.Size([150, 1])
epoch 12, loss 0.25031498074531555
torch.Size([150, 1])
epoch 13, loss 0.2358156144618988
torch.Size([150, 1])
epoch 14, loss 0.22228525578975677
torch.Size([150, 1])
epoch 15, loss 0.20966148376464844
torch.Size([150, 1])
epoch 16, loss 0.1978866308927536
torch.Size([150, 1])
epoch 17, loss 0.18690747022628784
torch.Size([150, 1])
epoch 18, loss 0.17667445540428162
torch.Size([150, 1])
epoch 19, loss 0.167141392827034
torch.Size([150, 1])
epoch 20, loss 0.15826515853405
torch.Size([150, 1])
epoch 21, loss 0.15000516176223755
torch.Size([150, 1])
epoch 22, loss 0.14232327044010162
torch.Size([150, 1])
epoch 23, loss 0.1351836770772934
torch.Size([150, 1])
epoch 24, loss 0.12855246663093567
torch.Size([150, 1])
epoch 25, loss 0.12239766120910645
torch.Size([150, 1])
epoch 26, loss 0.11668909341096878
torch.Size([150, 1])
epoch 27, loss 0.1113981381058693
torch.Size([150, 1])
epoch 28, loss 0.10649775713682175
torch.Size([150, 1])
epoch 29, loss 0.10196246951818466
```

```

torch.Size([150, 1])
epoch 30, loss 0.09776805341243744
torch.Size([150, 1])
epoch 31, loss 0.09389173239469528
torch.Size([150, 1])
epoch 32, loss 0.09031195193529129
torch.Size([150, 1])
epoch 33, loss 0.08700835704803467
torch.Size([150, 1])
epoch 34, loss 0.08396177738904953

```

Visualize and Evaluate Model

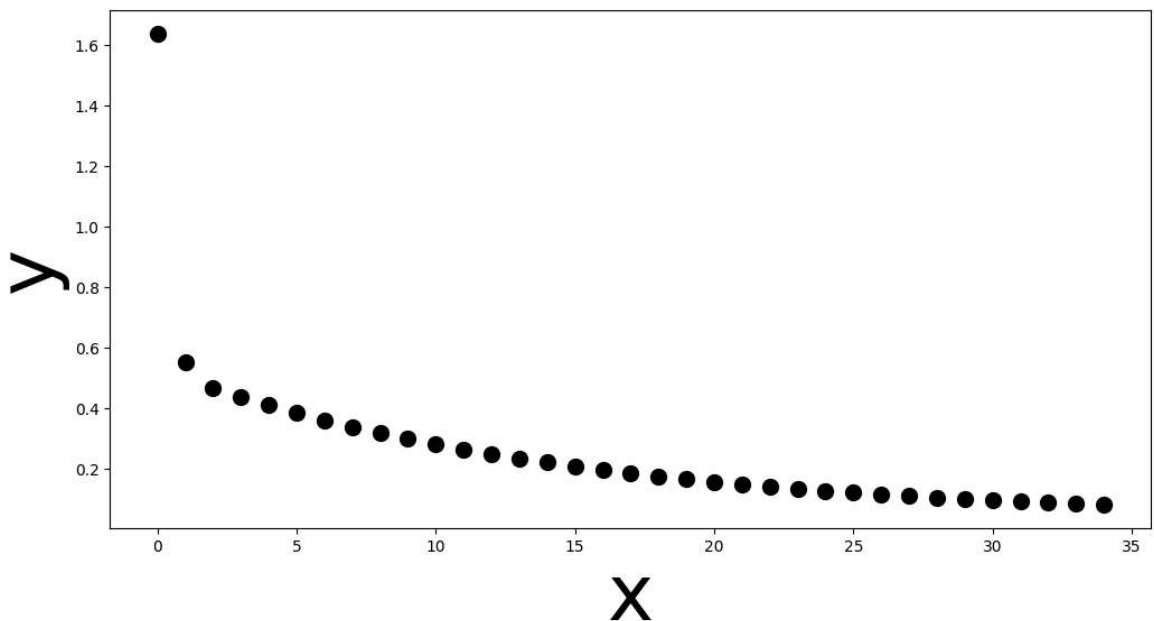
```

In [21]: # Plot your training loss throughout the training
# Include proper x and y labels for the plot

plt.figure(figsize=(12, 6))
plt.plot(train_loss_list, 'o', color = 'black', markersize = 10)
plt.xlabel('x', fontsize = 50)
plt.ylabel('y', fontsize = 50)
# YOUR CODE HERE

```

Out[21]: Text(0, 0.5, 'y')



```

In [42]: # Confirm that your model's training accuracy is >90%

with torch.no_grad():
    predicted = model(X_train)

    # print(predicted)
    # Compare your model predictions with targets (y_train) to compute the train
    # YOUR CODE HERE
    predicted = torch.round(predicted)
    print(predicted)
    # there is a rounding function with numpy array.

    t_f_results = predicted == y_train
    print(t_f_results.sum())
    accuracy = t_f_results.sum()/t_f_results.size(0)

```

```
print(accuracy)
# Training accuracy = (# of correct predictions) / (total # of training samples)
# You can round the model predictions to integer (e.g. 0.34 -> 0, 1.78 -> 2)

# YOUR CODE HERE
```

[illegible]

[illegible]

```
[2.],  
[2.],  
[2.],  
[1.],  
[2.],  
[2.],  
[1.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[1.],  
[1.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[2.],  
[1.],  
[2.],  
[2.],  
[2.]])
```

```
tensor(142)  
tensor(0.9467)  
torch.Size([150, 1])
```

In []: